

Proyecto Final – Avance 1: Documento de Diseño

Sistema de Facturación e Inventario Fidecompro (Proyecto 1) · Versión 15

[Nombre completo] · Carné: [número]

Universidad Fidélitas

Programación Cliente-Servidor Concurrente

Profesor: Mario Alberto Vargas Montes

8 de octubre de 2026

Tabla 1*Control de versiones del documento*

Versión	Fecha	Cambios
v1	06/10/2026	Primera versión: clases, 15 historias de usuario, 8 prototipos y diagramas de arquitectura, secuencia y navegación
v2	08/10/2026	Modelo de clases ajustado al material de las semanas 1 a 5 (estilo MultiSports); tabla de temas del curso; hilos, sockets, Swing y BD como arquitectura prevista; historias y prototipo de productos por tipo
v3	08/10/2026	Se agrega esta tabla de control de versiones y la versión en el nombre del archivo
v4	08/10/2026	Historias de usuario en formato de tabla con escenarios de aceptación (Dado que / Cuando / Resultado esperado)
v5	08/10/2026	Se agregan las listas de requerimientos funcionales (10) y no funcionales (8); las secciones siguientes se renumeran
v6	08/10/2026	Diseño alineado con lo que el profesor pide para el proyecto final: CRUD completo (con eliminar) desde pantallas, ventana del servidor (HU 16 y P9), base de datos MySQL o Derby con su modelo de tablas y plan de demostración para el video final
v7	08/10/2026	Guía al inicio con las tres partes que pide el Avance 1 (historias de usuario, diagrama de clases y bocetos); tabla de pantallas CRUD por clase; nuevo boceto P10 de categorías; nombre del profesor en la portada
v8	08/10/2026	Cada colección según su uso, como en los ejemplos del profesor: List donde importa el orden, Set para advertencias sin repetir, HashMap para usuarios y clientes por id, TreeMap para facturas ordenadas por número y CopyOnWriteArrayList para los clientes conectados al servidor; nueva tabla «Colecciones: cuál se usa y por qué» en la sección 4.2
v9	08/10/2026	La tabla «Colecciones: cuál se usa y por qué» cita el ejemplo del profesor que respalda cada elección (EjemploList , EjemploSet , EjemploMap , EjemploTreeMap , EjemploCollection) y explica, con EjemploColeccionGenerica y EjemploColeccionNoGenerica , por qué todas las colecciones son genéricas
v10	08/10/2026	La sección 4.1 abre con la vista general del diagrama de clases en una página completa, con su simbología (la misma imagen del diagrama simplificado v4 de draw.io, ya con las colecciones), y menciona los archivos editables de draw.io; la tabla del inicio indica dónde está
v11	08/10/2026	Formato APA 7: márgenes de 2,54 cm (también en las páginas horizontales de las historias), Times New Roman 12, interlineado doble con sangría en el texto, número de página arriba a la derecha, portada de estudiante, títulos de nivel 1 a 3 al estilo APA y tablas y figuras numeradas con su título en cursiva arriba
v12	08/10/2026	Historias de usuario de 16 a 12, una por cada pantalla CRUD: se unen iniciar y cerrar sesión (HU 1), registrar, editar y eliminar clientes (HU 3) y consultar y anular facturas (HU 11), conservando todos los escenarios; se renumeran las referencias en requerimientos, bocetos, trazabilidad y plan de demostración
v13	08/10/2026	Escenarios de aceptación consolidados: como máximo 4 por historia (de 65 a 40), uniendo los casos parecidos (por ejemplo, todos los datos inválidos de un formulario en un solo escenario) sin perder ninguna regla
v14	09/10/2026	Vista general del diagrama de clases más fácil de leer: cada tipo de relación con su color y una palabra sobre la línea (es un, implementa, contiene, agrupa, usa), multiplicidad también en palabras y simbología reordenada empezando por cómo leer las líneas (diagrama simplificado v5)
v15	09/10/2026	La identificación (tipo y número de cédula) pasa de Cliente a Persona , así que también la tienen los usuarios: diagramas de clases, tablas de atributos, modelo de base de datos (tabla USUARIOS), historia HU 2 y boceto P8 de usuarios

1. Introducción

La cadena de venta al por mayor **Fidecompro** necesita una aplicación de escritorio para llevar el inventario de sus productos y emitir facturas a sus clientes. Este documento presenta el diseño de la solución para el Avance 1 del proyecto final: las clases identificadas (con atributos, métodos y relaciones), las funcionalidades expresadas como historias de usuario y los prototipos de la interfaz gráfica.

Contenido del Avance 1. El Avance 1 es conceptual y pide tres partes. Esta tabla indica dónde está cada una; el resto del documento (requerimientos, arquitectura, base de datos y plan de demostración) las complementa.

Tabla 2

Partes del Avance 1 y dónde se encuentran

Parte pedida	Dónde está	Qué contiene
Historias de usuario	Sección 5	12 historias con su rol, funcionalidad, finalidad y escenarios de aceptación
Diagrama de clases	Secciones 4.1 y 4.3	Vista general en una página con su simbología (4.1), diagramas detallados con atributos, métodos y relaciones (4.1 y 4.3); la sección 4.2 describe cada clase
Bocetos (mockups) de las pantallas	Sección 6	Inicio de sesión, menú principal y pantallas CRUD de cada clase (clientes, productos, categorías, usuarios y facturas), más la ventana del servidor

1.1 Requerimientos del enunciado

Tabla 3

Requerimientos del enunciado y las historias de usuario que los cubren

#	Requerimiento del enunciado	Historias de usuario que lo cubren
R1	Creación de los registros de los clientes	HU 3
R2	Registro de los productos (varios tipos de productos)	HU 4, HU 5, HU 6 y HU 7
R3	Crear facturas a los clientes	HU 8, HU 9 y HU 11
R4	Entregar la factura física (archivo con el desglose de pago)	HU 10
R5	Ingreso con usuario y contraseña	HU 1 y HU 2

1.2 Requisitos del proyecto final

El profesor indicó que el proyecto final debe cumplir cuatro requisitos técnicos y demostrarse en un video. Aunque varios temas todavía no se han visto en el curso, el diseño de este avance ya los contempla para no tener que rehacerlo después:

Tabla 4

Requisitos del proyecto final y cómo los cumple el diseño

Requisito del proyecto final	Cómo lo cumple el diseño	Dónde
Cliente-servidor con sockets	Cada caja es una aplicación cliente que se conecta por Socket al ServerSocket del servidor (puerto 5000) e intercambia objetos Solicitud y Respuesta	Secciones 4.3 y 4.4
Hilos	El servidor atiende cada cliente conectado en su propio hilo (ManejadorCliente); las operaciones que modifican datos se sincronizan	Secciones 4.4 y 4.5
Pantallas JFrame	Ventanas Swing para el cliente (P1 a P8 y P10) y una ventana propia del servidor (P9)	Sección 6
Base de datos MySQL o Derby	Todo se guarda en la base de datos por medio de JDBC y clases DAO	Sección 4.6
CRUD completo desde pantallas	Crear, consultar, modificar y eliminar usuarios, clientes, categorías y productos; crear, consultar y anular facturas	Secciones 2 y 8
Varios clientes a la vez	Se pueden abrir varias ventanas cliente al mismo tiempo y el servidor las atiende en paralelo	HU 9 y HU 12
Video final	Guion de demostración de todas las funcionalidades	Sección 8

1.3 Alcance y decisiones técnicas

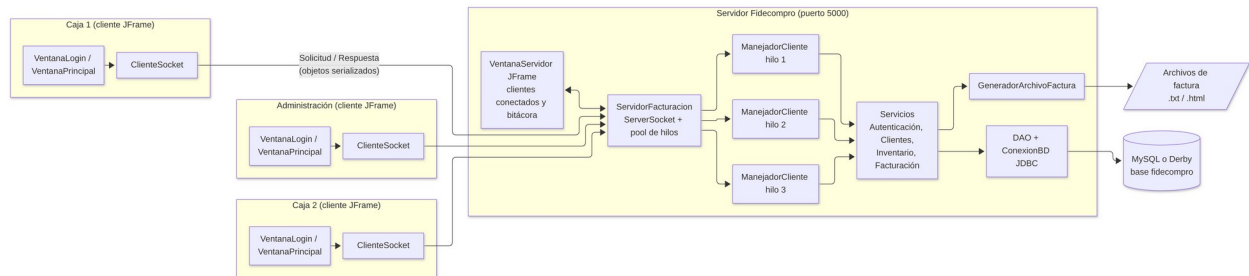
- **Lenguaje y entorno:** Java 17, proyecto Maven en NetBeans IDE, interfaz gráfica con **Java Swing** (ventanas **JFrame** diseñadas en NetBeans). No se usa ningún framework; todo el código se escribe desde cero.
- **Base del diseño:** el modelo de clases (sección 4) aplica lo visto en las semanas 1 a 5: herencia, clases abstractas, interfaces, polimorfismo, colecciones y excepciones propias. Hilos, sockets, pantallas **JFrame** y base de datos aún no se han visto en el curso; se incluyen como **arquitectura prevista**, porque son obligatorios en el proyecto final, y se detallarán en los siguientes avances.
- **Arquitectura cliente-servidor:** un **servidor** central atiende a varias cajas (clientes **JFrame**) al mismo tiempo mediante **sockets TCP**. Cada conexión es atendida por su propio hilo (**ManejadorCliente**) dentro de un pool de hilos. El servidor tiene su propia

ventana (*VentanaServidor*) para iniciarlo, detenerlo y ver los clientes conectados y cada operación que atiende.

- **Concurrencia:** cuando dos vendedores facturan el mismo producto a la vez, el servidor valida y descuenta el stock dentro de un bloque *synchronized*, de modo que nunca se venda más inventario del que existe. El número de factura sale del contador *static Factura.ultimoNumero*, que se incrementa dentro de un método *synchronized* para que dos facturas nunca reciban el mismo número. Las demás operaciones que modifican datos (crear, modificar y eliminar) también se sincronizan en el servidor. En el cliente, las consultas largas se ejecutan con *SwingWorker* para no congelar la ventana, y cada pantalla tiene el botón *Actualizar lista* para ver los cambios hechos desde otras cajas.
- **Persistencia:** base de datos **MySQL** accedida con JDBC (API estándar de Java) a través de clases DAO. Como alternativa se puede usar **Apache Derby**, la otra base de datos que se verá en el curso: las sentencias SQL son las mismas y solo cambia la URL de conexión en *ConexionBD*.
- **Eliminar sin perder historial:** usuarios, clientes, categorías y productos se pueden eliminar desde su pantalla. Si un registro ya está en una factura (un cliente con compras, por ejemplo), el sistema no lo borra para no dañar el historial; lo desactiva y lo explica al usuario.
- **Factura física:** el servidor genera un archivo *.txt* (y opcionalmente *.html*) con el desglose completo de la factura, que el usuario guarda o imprime.
- **Roles:** ADMINISTRADOR (gestiona usuarios, catálogo e inventario, puede anular facturas) y VENDEDOR (registra clientes y emite facturas).

Figura 1

Arquitectura general de la solución



2. Lista de requerimientos funcionales

Los requerimientos funcionales describen lo que el sistema debe hacer. Cada uno indica las historias de usuario que lo detallan (sección 5).

Tabla 5*Requerimientos funcionales*

#	Requerimiento	Descripción	Prioridad
1	El sistema debe permitir crear, consultar, modificar y eliminar usuarios, y que ingresen con usuario y contraseña.	Solo el personal autorizado puede usar el sistema. Cada usuario tiene un rol (administrador o vendedor) que define a qué módulos entra. Un usuario con facturas emitidas no se elimina para no perder el historial; en ese caso se desactiva. Es el primer requerimiento a implementar porque todas las demás funciones dependen de saber quién está trabajando. Historias: HU 1 y HU 2.	Alta
2	El sistema debe permitir crear, consultar, modificar y eliminar clientes.	Los clientes son la base de la facturación: cada factura se emite a uno de ellos. Incluye tipo y número de identificación, nombre o razón social, teléfono, correo y dirección. Un cliente con facturas no se puede eliminar; en ese caso se desactiva y conserva su historial. Historia: HU 3.	Alta
3	El sistema debe permitir crear, consultar, modificar y eliminar categorías de productos.	Las categorías (abarrotes, bebidas, limpieza, higiene...) ordenan el catálogo y facilitan encontrar productos al facturar. Una categoría con productos no se puede eliminar. Historia: HU 4.	Media
4	El sistema debe permitir crear, consultar, modificar y eliminar productos de distintos tipos.	La cadena maneja varios tipos de productos (abarrotes, bebida y artículo del hogar), cada uno con datos propios y su propio porcentaje de IVA. Incluye código, nombre, valor de compra, valor de venta, existencias y stock mínimo. Un producto que ya aparece en facturas no se elimina; se desactiva. Historia: HU 5.	Alta
5	El sistema debe permitir registrar entradas de mercadería y ajustes de existencias.	Mantiene el inventario del sistema igual al de bodega. Cada ajuste indica su motivo y nunca puede dejar existencias en negativo. Historia: HU 6.	Alta
6	El sistema debe alertar sobre los productos que están por debajo del stock mínimo.	Permite reabastecer a tiempo y evitar ventas perdidas. La alerta aparece en el menú principal y en una lista ordenada de menos a más existencias. Historia: HU 7.	Media
7	El sistema debe permitir crear facturas a los clientes con uno o varios productos.	Calcula subtotal, descuento, IVA por línea según el tipo de producto y total; asigna un número consecutivo único y descuenta las existencias al emitir. Es el proceso central del negocio. Historias: HU 8 y HU 9.	Alta
8	El sistema debe generar la factura física en un archivo con el desglose de pago.	El enunciado pide entregar la factura física. Se genera un archivo con los datos de la empresa, del cliente, cada línea, subtotal, descuento, IVA, total y método de pago, que se puede imprimir o enviar. Historia: HU 10.	Alta
9	El sistema debe permitir consultar el historial de facturas por fechas, cliente y estado.	Permite dar seguimiento a las ventas y conocer el total vendido en un periodo, contando solo las facturas emitidas. Historia: HU 11.	Media
10	El sistema debe permitir anular facturas emitidas.	Corrige ventas registradas por error. Al anular, la factura queda en estado ANULADA y las existencias de cada línea vuelven al inventario. Solo el administrador puede hacerlo. Historia: HU 11.	Media
11	El servidor debe tener su propia ventana para iniciarlo, detenerlo y ver qué clientes	Permite comprobar que el servidor atiende a varias cajas a la vez, cada una en su propio hilo, y que cada operación de las pantallas llega a la base de datos. Historia: HU 12.	Alta

#	Requerimiento	Descripción	Prioridad
	están conectados y qué operaciones atiende.		

3. Lista de requerimientos no funcionales

Los requerimientos no funcionales describen cómo debe comportarse el sistema y las condiciones técnicas que debe cumplir.

Tabla 6

Requerimientos no funcionales

#	Requerimiento	Descripción	Prioridad
1	El sistema debe desarrollarse en Java 17 con NetBeans, sin frameworks ni código previo.	Es una condición del enunciado del proyecto: todo el código se escribe desde cero, usando solo las bibliotecas estándar de Java (Swing, sockets, hilos y JDBC).	Alta
2	El sistema debe contar con pantallas gráficas de escritorio en español hechas con JFrame.	Las ventanas del cliente y del servidor se construyen con Java Swing (JFrame) en NetBeans. Siguen un mismo estilo, muestran mensajes claros y marcan los campos con error, para que el personal de las cajas las use sin capacitación extensa. Todo el CRUD se hace desde las pantallas.	Alta
3	El sistema debe funcionar con una arquitectura cliente-servidor por sockets.	Se pueden abrir varias ventanas cliente al mismo tiempo, en la misma computadora o en varias, y todas se conectan por sockets TCP a un servidor central que guarda la información. Así todas las cajas trabajan sobre los mismos datos de clientes, productos y facturas.	Alta
4	El sistema debe soportar varios clientes haciendo operaciones al mismo tiempo sin generar inconsistencias.	El servidor atiende cada conexión en un hilo distinto. Las operaciones que modifican datos (crear, modificar, eliminar, descontar existencias y numerar facturas) se sincronizan, para que nunca se venda más producto del que hay ni se repita un número de factura. Cada pantalla tiene el botón "Actualizar lista" para ver los cambios hechos desde otras cajas.	Alta
5	El sistema debe proteger el acceso y la información de los usuarios.	Las contraseñas se guardan cifradas, el ingreso se bloquea por un minuto tras tres intentos fallidos y cada rol solo ve los módulos que le corresponden.	Alta
6	El sistema debe almacenar la información en una base de datos MySQL o Derby.	Usuarios, clientes, categorías, productos y facturas se guardan en MySQL (o en Apache Derby, la otra base de datos que se verá en el curso) a través de JDBC, de forma ordenada y sin perderse al cerrar la aplicación. Solo la clase ConexionBD cambia si se pasa de una a otra.	Alta
7	El sistema debe controlar los errores sin dejar datos incompletos.	Los errores se manejan con excepciones propias y mensajes claros. Una operación que falla (por ejemplo, una factura sin stock suficiente) no deja cambios a medias en el inventario.	Alta
8	El sistema debe responder rápido y sin congelar la pantalla.	Las operaciones frecuentes, como buscar un cliente o emitir una factura, deben responder en menos de 2 segundos. Las consultas largas se ejecutan en segundo plano para que la ventana siga respondiendo.	Media

4. Entidades y clases

El modelo de clases sigue la forma de trabajo vista en las semanas 1 a 5 del curso, en particular la práctica integradora MultiSports de la semana 5:

Tabla 7

Conceptos del curso aplicados en el diseño

Concepto del curso	Dónde se aplica en Fidecompro
Clase abstracta con subclases que sobrescriben un método (semanas 2 y 3)	Producto → Abarrote, Bebida, ArtículoHogar; Persona → Usuario, Cliente
Interface terminada en <i>-able</i> con <code>mostrarInformacion()</code> (semanas 3 y 5)	Mostrable, implementada por personas, productos, facturas y detalles
Polimorfismo: distintos objetos responden distinto al mismo mensaje (semana 3)	Cada tipo de producto responde <code>obtenerPorcentajeImpuesto()</code> a su manera
Atributos <code>static</code> e IDs autoincrementales (semanas 1 y 5)	<code>idAutoIncremental</code> en Producto, Categoria, Cliente y Usuario; <code>Factura.ultimoNumero</code>
Constantes <code>final</code> (semana 1, ejemplo del IVA)	<code>Producto.IVA_GENERAL = 0.13</code> y <code>Abarrote.IVA_CANASTA_BASICA = 0.01</code>
Composición y agregación con colecciones genéricas (semanas 1 y 5)	<code>Factura</code> contiene sus <code>DetalleFactura</code> ; <code>Inventario</code> agrupa <code>Categoria</code> y <code>Categoria</code> agrupa <code>Producto</code>
Colecciones <code>List</code> , <code>Set</code> y <code>Map</code> (<code>HashMap</code> , <code>TreeMap</code>) declaradas por su interface, cada una según su uso (semana 4)	<code>List</code> en detalles, productos y categorías; <code>Set</code> en advertencias; <code>Map</code> en los registros de usuarios, clientes y facturas (tabla «Colecciones» en la sección 4.2)
Métodos <code>agregarX</code> / <code>editarX</code> / <code>eliminarX</code> / <code>mostrarX</code> por id (semana 5)	<code>Inventario</code> , <code>Categoria</code> , <code>RegistroClientes</code> , <code>RegistroUsuarios</code> , <code>RegistroFacturas</code>
Enumeraciones para catálogos cerrados (semana 1)	<code>Rol</code> , <code>TipoIdentificacion</code> , <code>MetodoPago</code> , <code>EstadoFactura</code>
<code>Comparable</code> y <code>Comparator</code> (semana 4)	<code>Producto</code> implements <code>Comparable<Producto></code> (orden por nombre) y <code>ComparadorPorExistencias</code>
Excepciones propias y <code>try/catch/finally</code> (semana 4)	<code>StockInsuficienteException</code> , <code>CredencialesInvalidasException</code> , <code>RegistroNoEncontradoException</code> ; <code>IOException</code> al escribir la factura

Las clases se organizan en paquetes de un proyecto Maven de NetBeans:

Tabla 8

Paquetes del proyecto

Paquete	Contenido
<code>com.fidecompro.modelo</code>	Entidades, enumeraciones, interface <code>Mostrable</code> y clases que administran colecciones (<code>Inventario</code> , <code>Registro...</code>)
<code>com.fidecompro.excepciones</code>	Excepciones propias del negocio
<code>com.fidecompro.servidor</code>	Servidor de sockets, hilos de atención y servicios (arquitectura prevista, sección 4.3)
<code>com.fidecompro.datos</code>	Conexión a la base de datos y clases DAO (arquitectura prevista)

Paquete	Contenido
com.fidecompro.cliente	Conexión del cliente y ventanas Swing (arquitectura prevista)

4.1 Diagramas de clases del modelo de dominio

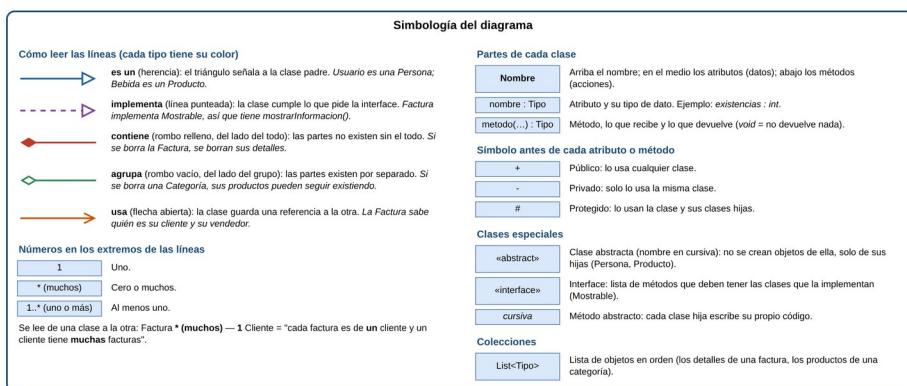
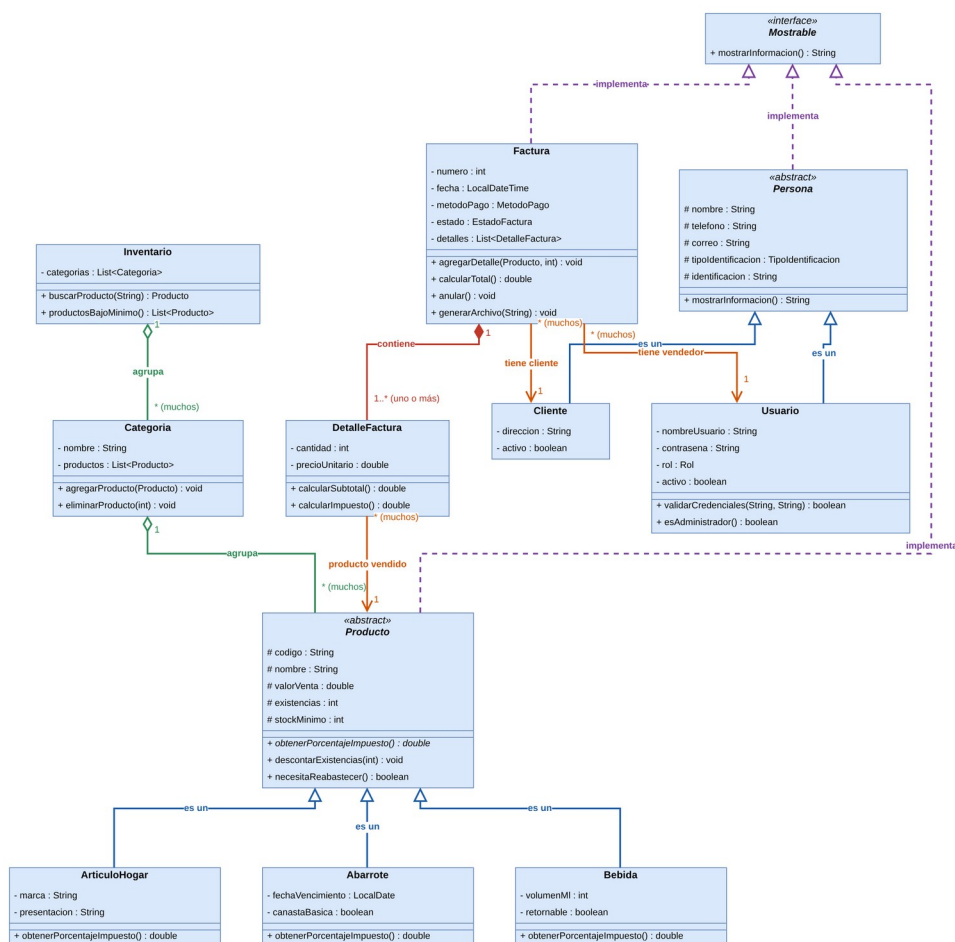
Primero se muestra una vista general en una sola página con las 12 clases principales, sus atributos y métodos más importantes, sus relaciones y una simbología que explica cómo leerlo: cada tipo de relación tiene su color y una palabra sobre la línea (*es un*, *implementa*, *contiene*, *agrupa*, *usa*), y la multiplicidad se escribe también en palabras (*muchos*, *uno o más*). Después, el detalle completo en dos diagramas para que se lea con claridad. Todos los diagramas de clases se entregan también en draw.io para editarlos

(Avance1_Diagrama_de_Clases_Simplificado_v6.drawio y Avance1_Diagrama_de_Clases_v5.drawio).

Vista general del modelo de dominio

Figura 2

Diagrama de clases simplificado con simbología



Detalle del modelo de dominio

El primer diagrama detallado cubre el inventario, los tipos de producto y las excepciones; el segundo, las personas y la facturación.

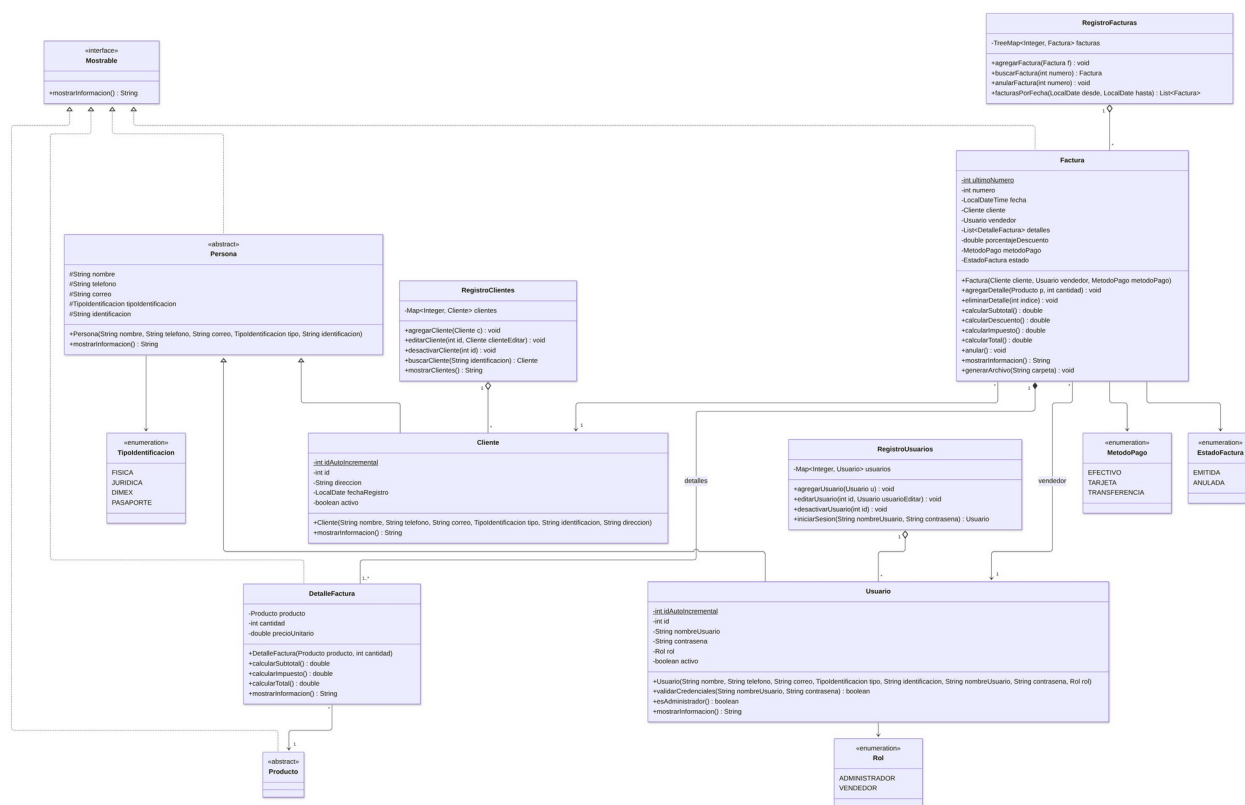
Figura 3

Diagrama de clases: inventario, tipos de producto y excepciones



Figura 4

Diagrama de clases: personas y facturación



4.2 Descripción de las clases del dominio

Todas las clases tienen constructor con sus campos, *getters* y *setters*. En las tablas se listan solo los métodos con lógica propia.

Mostrable (interface)

Contrato que obliga a cada objeto a describirse a sí mismo como texto. Se usa para listar en pantalla y para armar la factura física.

Tabla 9

Métodos de la interface Mostrable

Método	Descripción
mostrarInformacion(): String	Devuelve la información del objeto lista para mostrar o imprimir

Persona (abstracta)

Datos comunes de quien usa el sistema y de a quien se le vende. No se instancia directamente. Implementa **Mostrable**.

Tabla 10

Atributos de la clase Persona

Atributo	Tipo	Descripción
nombre	String	Nombre completo o razón social
telefono	String	Teléfono de contacto
correo	String	Correo electrónico
tipoIdentificacion	TipoIdentificacion	Física, jurídica, DIMEX o pasaporte
identificacion	String	Número de cédula o documento (único entre los usuarios y entre los clientes)

Relaciones: superclase de **Usuario** y **Cliente** (herencia jerárquica).

Usuario (extends Persona)

Persona que ingresa al sistema: vendedor o administrador.

Tabla 11

Atributos de la clase Usuario

Atributo	Tipo	Descripción
idAutoIncremental	static int	Contador de la clase para asignar el siguiente id
id	int	Identificador asignado en el constructor
nombreUsuario	String	Nombre con el que inicia sesión (único)
contrasena	String	Contraseña de acceso
rol	Rol	ADMINISTRADOR o VENDEDOR
activo	boolean	Un usuario inactivo no puede ingresar

Tabla 12

Métodos de la clase Usuario

Método	Descripción
validarCredenciales(String, String): boolean	Compara usuario y contraseña con los suyos y revisa que esté activo
esAdministrador(): boolean	Indica si puede entrar a los módulos restringidos
mostrarInformacion(): String	Sobrescribe el de Persona agregando usuario y rol

Relaciones: emite muchas **Factura** (1 a *); lo administra **RegistroUsuarios**.

Cliente (extends Persona)

Comercio o persona a quien Fidecompro le vende.

Tabla 13***Atributos de la clase Cliente***

Atributo	Tipo	Descripción
idAutoIncremental	static int	Contador de la clase
id	int	Identificador
direccion	String	Dirección de entrega
fechaRegistro	LocalDate	Fecha en que se creó el registro
activo	boolean	Los clientes inactivos no aparecen al facturar

Tabla 14***Métodos de la clase Cliente***

Método	Descripción
mostrarInformacion(): String	Sobrescribe el de <i>Persona</i> agregando id y dirección

Relaciones: tiene muchas *Factura* (1 a *); lo administra *RegistroClientes*.

Producto (abstracta)

Representa cualquier artículo que vende la cadena. Como el enunciado indica que Fidecompro maneja **varios tipos de productos**, cada tipo es una subclase, igual que *Pelota*, *Bate* y *EquipoDeAnotacion* en la práctica *MultiSports*. Implementa *Mostrable* y *Comparable<Producto>*.

Tabla 15***Atributos de la clase Producto***

Atributo	Tipo	Descripción
IVA_GENERAL	static final double	Constante 0.13 (tarifa general del IVA)
idAutoIncremental	static int	Contador de la clase
id	int	Identificador
codigo	String	Código interno único (p. ej. ABR-0012)
nombre, descripcion	String	Datos descriptivos
valorCompra	double	Costo al que Fidecompro compra el producto

Atributo	Tipo	Descripción
valorVenta	double	Precio de venta sin impuesto
existencias	int	Unidades disponibles
stockMinimo	int	Nivel a partir del cual se alerta reabastecer
categoria	Categoria	Categoría a la que pertenece

Tabla 16

Métodos de la clase Producto

Método	Descripción
obtenerPorcentajeImpuesto(): double	Abstracto. Cada tipo de producto define su impuesto (polimorfismo)
descontarExistencias(int): void	Resta unidades; lanza <code>StockInsuficienteException</code> si no alcanzan
aumentarExistencias(int): void	Suma unidades (compras, ajustes o facturas anuladas)
necesitaReabastecer(): boolean	<code>existencias <= stockMinimo</code>
compareTo(Producto): int	Orden natural por nombre, para listar el catálogo con <code>Collections.sort</code>
mostrarInformacion(): String	Código, nombre, precio y existencias

Relaciones: superclase de `Abarrote`, `Bebida` y `ArticuloHogar`; agregado en

`Categoria` (* a 1); referenciado por `DetalleFactura`.

Abarrote, Bebida y ArticuloHogar (extends Producto)

Tabla 17

Subclases de Producto

Subclase	Atributos propios	Impuesto (obtenerPorcentajeImpuesto)	Métodos propios
Abarrote (arroz, frijoles, aceite, café...)	<code>fechaVencimiento: LocalDate</code> , <code>pesoKg: double</code> , <code>canastaBasica: boolean</code> ; constante <code>IVA_CANASTA_BASICA = 0.01</code>	1 % si es canasta básica, si no IVA_GENERAL	<code>estaVencido(): boolean</code>
Bebida (refrescos, jugos, agua)	<code>volumenMl: int</code> , <code>unidadesPorPaquete: int</code> , <code>retornable: boolean</code>	IVA_GENERAL	—
ArticuloHogar (limpieza e higiene)	<code>marca: String</code> , <code>presentacion: String</code> , <code>advertencias: Set<String></code> (sin repetidas)	IVA_GENERAL	<code>agregarAdvertencia(String)</code> , <code>eliminarAdvertencia(String)</code>

Las tres sobrescriben `mostrarInformacion()` llamando a `super.mostrarInformacion()` y agregando sus datos propios.

Categoria

Agrupar productos para ordenar el catálogo (Abarrotes, Bebidas, Limpieza, Higiene...), igual que en MultiSports.

Tabla 18

Atributos de la clase Categoria

Atributo	Tipo	Descripción
idAutoIncremental	static int	Contador de la clase
id	int	Identificador
nombre, descripcion	String	Datos de la categoría
productos	List<Producto>	Productos de la categoría, en orden

Tabla 19

Métodos de la clase Categoria

Método	Descripción
agregarProducto(Producto): void	Agrega un producto y le asigna esta categoría
editarProducto(int idProducto, Producto): void	Reemplaza los datos del producto con ese id
eliminarProducto(int idProducto): void	Quita el producto; lanza <code>RegistroNoEncontradoException</code> si no existe
buscarProducto(int idProducto): Producto	Busca por id
mostrarProductos(): String	Une el <code>mostrarInformacion()</code> de cada producto

Relaciones: agregación con Producto (1 a *); agregada en Inventario.

Inventario

Punto central del catálogo de una sede.

Tabla 20

Atributos de la clase Inventario

Atributo	Tipo	Descripción
nombre	String	Nombre del inventario
sede	String	Sucursal de Fidecompro
categorias	List<Categoria>	Categorías registradas

Tabla 21*Métodos de la clase Inventario*

Método	Descripción
agregarCategoria / editarCategoria / eliminarCategoria	CRUD de categorías por id
agregarProductoCategoria(int idCategoria, Producto)	Agrega un producto en la categoría indicada
editarProductoCategoria / eliminarProductoCategoria	Edita o elimina un producto dentro de una categoría
mostrarProductosCategoria(int idCategoria): String	Lista los productos de una categoría
buscarProducto(String codigo): Producto	Busca en todas las categorías; se usa al facturar
productosBajoMinimo(): List<Producto>	Productos con <code>necesitaReabastecer()</code> , ordenados con <code>ComparadorPorExistencias</code>

ComparadorPorExistencias (implements Comparator<Producto>)

Orden alterno al natural: de menos a más existencias, para mostrar primero lo que urge reabastecer.

Factura

Documento de venta emitido a un cliente. Implementa `Mostrable`.

Tabla 22*Atributos de la clase Factura*

Atributo	Tipo	Descripción
ultimoNumero	static int	Último consecutivo usado (como <code>Factura.ultimoNumero</code> en el ejemplo de la semana 1)
numero	int	Consecutivo de esta factura (se muestra como <code>FC-000123</code>)
fecha	LocalDateTime	Fecha y hora de emisión
cliente	Cliente	A quién se factura
vendedor	Usuario	Quién la emite
detalles	List<DetalleFactura>	Líneas de la factura, en el orden en que se agregan
porcentajeDescuento	double	Descuento aplicado al subtotal
metodoPago	MetodoPago	Efectivo, tarjeta o transferencia
estado	EstadoFactura	EMITIDA o ANULADA

Tabla 23*Métodos de la clase Factura*

Método	Descripción
agregarDetalle(Producto, int): void	Crea la línea; si el producto ya está, suma la cantidad. Lanza <code>StockInsuficienteException</code>

Método	Descripción
eliminarDetalle(int): void	Quita una línea
calcularSubtotal / calcularDescuento / calcularImpuesto / calcularTotal	Recorren detalles y suman; el impuesto de cada línea lo da su producto (polimorfismo)
anular(): void	Cambia el estado a ANULADA
mostrarInformacion(): String	Arma el desglose completo de la factura como texto
generarArchivo(String carpeta): void	Escribe <code>mostrarInformacion()</code> en <code>FC-000123.txt</code> con <code>FileWriter</code> , manejando <code>IOException</code> con <code>try/catch/finally</code>

Relaciones: composición con `DetalleFactura` (1 a 1..*): las líneas no existen sin su factura. Asociación con `Cliente` y `Usuario` (* a 1). La administra `RegistroFacturas`.

DetalleFactura

Línea de una factura. Implementa `Mostrable`.

Tabla 24

Atributos de la clase DetalleFactura

Atributo	Tipo	Descripción
producto	Producto	Producto vendido
cantidad	int	Unidades
precioUnitario	double	Copia del <code>valorVenta</code> al momento de la venta, para conservar el histórico

Tabla 25

Métodos de la clase DetalleFactura

Método	Descripción
calcularSubtotal(): double	<code>cantidad × precioUnitario</code>
calcularImpuesto(): double	<code>subtotal × producto.obtenerPorcentajeImpuesto()</code>
calcularTotal(): double	Subtotal más impuesto
mostrarInformacion(): String	Una línea del desglose: cantidad, descripción, precio y total

RegistroUsuarios, RegistroClientes y RegistroFacturas

Clases que administran las colecciones, con el mismo patrón de `Inventario`. Usuarios y clientes se guardan en un `Map` cuya clave es su id, y las facturas en un `TreeMap` cuya clave es el número, así buscar, editar o eliminar no necesita recorrer toda la colección.

Tabla 26*Clases de registro*

Clase	Atributo	Métodos
RegistroUsuarios	usuarios: Map<Integer, Usuario> (clave: id)	agregarUsuario, editarUsuario, eliminarUsuario, desactivarUsuario, iniciarSesion(nombreUsuario, contraseña) que devuelve el Usuario o lanza CredencialesInvalidasException
RegistroClientes	clientes: Map<Integer, Cliente> (clave: id)	agregarCliente (rechaza identificaciones repetidas), editarCliente, eliminarCliente, desactivarCliente, buscarCliente(identificacion), mostrarClientes
RegistroFacturas	facturas: TreeMap<Integer, Factura> (clave: número, ordenadas)	agregarFactura, buscarFactura(numero), anularFactura(numero) (devuelve las existencias al inventario), facturasPorFecha(desde, hasta), tieneFacturasCliente(idCliente) y tieneFacturasUsuario(idUsuario) (se consultan antes de eliminar)

*Excepciones propias (extends Exception)***Tabla 27***Excepciones propias*

Excepción	Cuándo se lanza
StockInsuficienteException	Se intenta facturar o descontar más unidades de las que hay
CredencialesInvalidasException	Usuario o contraseña incorrectos, o usuario inactivo
RegistroNoEncontradoException	Se busca, edita o elimina un id o identificación que no existe

Enumeraciones

- Rol: ADMINISTRADOR, VENDEDOR.
- TipoIdentificacion: FISICA, JURIDICA, DIMEX, PASAPORTE.
- MetodoPago: EFECTIVO, TARJETA, TRANSFERENCIA.
- EstadoFactura: EMITIDA, ANULADA.

Colecciones: cuál se usa y por qué

Como en los ejemplos de colecciones del profesor (semana 4), no todo se guarda en una lista: cada colección se elige según lo que se necesita (orden, duplicados o búsqueda por clave). Las variables se declaran por su interface y se crean con una implementación, igual que

```
List<String> lista = new ArrayList<>() en EjemploList, por ejemplo
```

```
List<DetalleFactura> detalles = new ArrayList<>() o Map<Integer,
```

```
Cliente> clientes = new HashMap<>().
```

Todas son genéricas, como en

EjemploColeccionGenerica: así no se necesita conversión de tipo (cast) al sacar un elemento, que es el problema que muestra EjemploColeccionNoGenerica.

Tabla 28

Colecciones utilizadas y por qué

Colección	Dónde se usa	Por qué	Ejemplo del curso
List (ArrayList)	Factura.detalles	Las líneas se muestran en el orden en que se agregan y se quitan por posición (eliminarDetalle(indice))	EjemploList: mantiene el orden de inserción y permite get y remove por índice
List (ArrayList)	Categoria.productos, Inventario.categorias y los resultados de listarProductos(), productosBajoMinimo() y facturasPorFecha()	Se recorren y se muestran en orden, y se ordenan con Collections.sort usando Comparable (por nombre) o Comparator (por existencias)	EjemploList y EjemploCollection (recorrido con for mejorado)
Set (HashSet)	ArticuloHogar.advertencias	Una advertencia no debe salir dos veces en el producto; el Set no acepta elementos repetidos	EjemploSet: HashSet ignora el elemento duplicado
Map (HashMap)	RegistroUsuarios.usuarios, RegistroClientes.clientes	El CRUD es por id: get(id), put(id, ...) y remove(id) llegan directo al registro sin recorrer la colección, y no puede haber dos con el mismo id (igual que la llave primaria en la base de datos)	EjemploMap: pares clave-valor con put, get, containsKey y remove
TreeMap	RegistroFacturas.facturas	Se busca la factura por su número y, además, el TreeMap las mantiene	EjemploTreeMap: claves ordenadas, primera y última entrada y rangos con

Colección	Dónde se usa	Por qué	Ejemplo del curso
		ordenadas por número: el historial sale en orden y <code>firstKey()/lastKey()</code> dan la primera y la última	<code>subMap</code>
<code>CopyOnWriteArrayList</code>	<code>ServidorFacturacion.clientesConectados</code>	Varios hilos agregan y quitan clientes conectados mientras la ventana del servidor recorre la lista; esta lista es segura para hilos	No está en los ejemplos: corresponde a hilos, que el curso verá más adelante

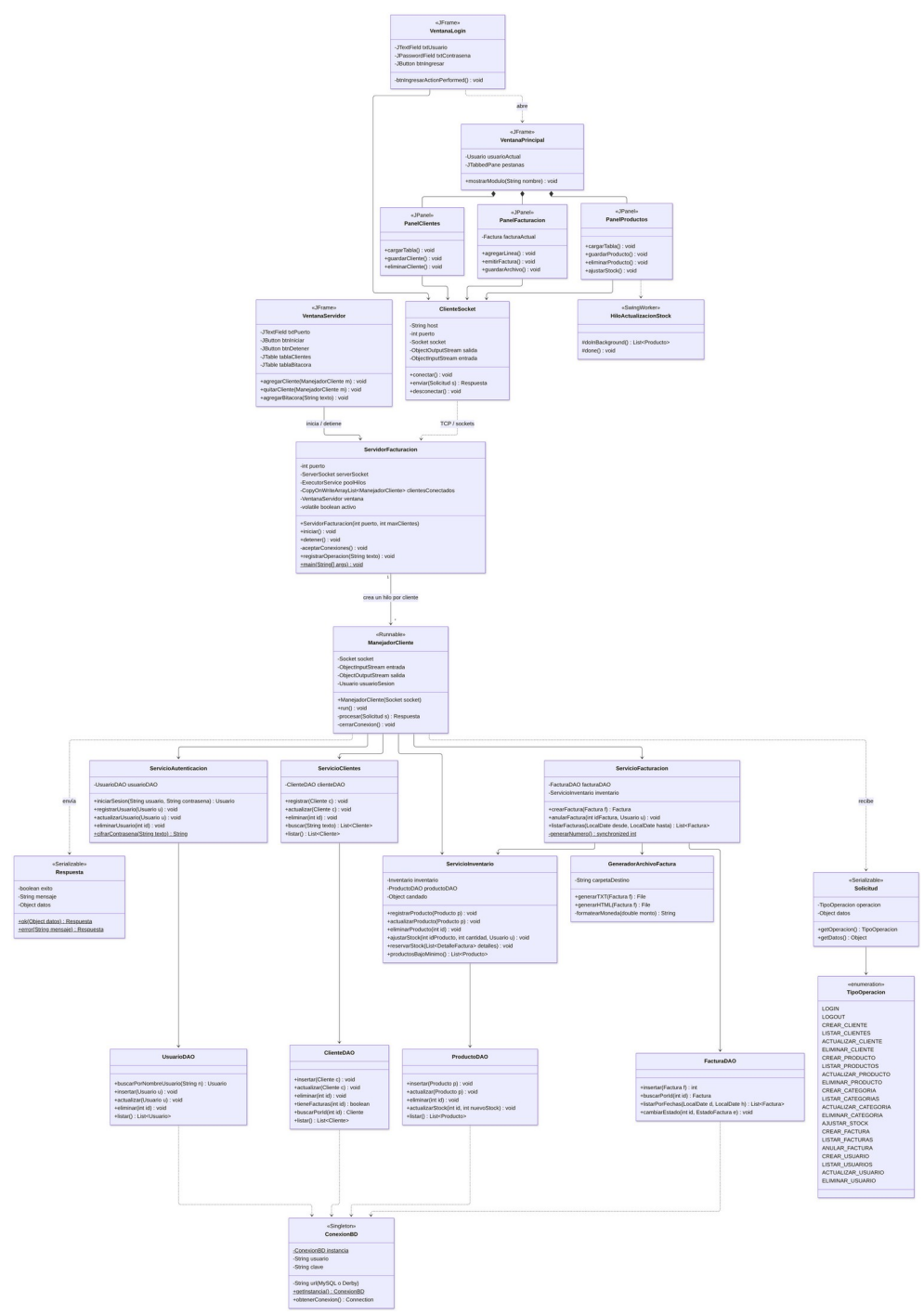
En el servidor, cuando varios hilos `ManejadorCliente` usen los mismos registros a la vez, los `HashMap` se crean como `ConcurrentHashMap` o se protegen con métodos `synchronized`, como ya se hace en `agregarFactura` y `anularFactura`.

4.3 Diagrama de clases cliente-servidor

Este diagrama es la **arquitectura prevista** para los siguientes avances, cuando el curso cubra hilos, sockets, pantallas `JFrame` y bases de datos. Muestra las clases que harán funcionar la aplicación en red y de forma concurrente sobre el modelo de la sección 4.1: las ventanas del cliente y del servidor, la conexión por sockets, el servidor con sus hilos, los servicios y el acceso a datos. La enumeración `TipoOperacion` incluye las cuatro operaciones del CRUD de cada entidad.

Figura 5

Diagrama de clases cliente-servidor



4.4 Descripción de las clases cliente-servidor

Tabla 29

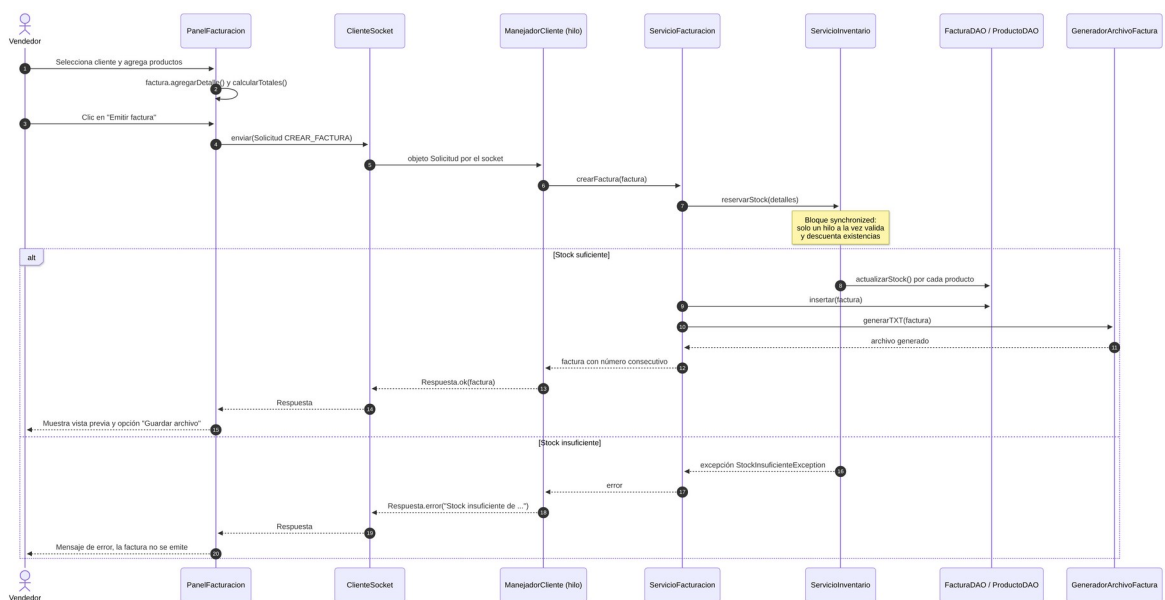
Clases cliente-servidor

Clase	Responsabilidad	Métodos principales	Relaciones
VentanaServidor (JFrame)	Pantalla del servidor: puerto, botones Iniciar y Detener, tabla de clientes conectados y bitácora de operaciones	agregarCliente(), quitarCliente(), agregarBitacora()	Inicia y detiene ServidorFacturacio n
ServidorFacturacion	Abre un <code>ServerSocket</code> en el puerto 5000 y acepta conexiones; cada conexión se entrega a un pool de hilos (<code>ExecutorService</code>). Lleva los clientes conectados en una <code>CopyOnWriteArrayList</code> (segura para hilos) y avisa a la ventana de cada operación	iniciar(), detener(), aceptarConexiones() , registrarOperacion (), main()	Crea muchos ManejadorCliente (1 a *)
ManejadorCliente (implementa <code>Runnable</code>)	Atiende a un cliente conectado en su propio hilo: lee <code>Solicitud</code> , llama al servicio correspondiente y responde con <code>Respuesta</code> . Guarda el usuario de la sesión para validar permisos	run(), procesar(<code>Solicitud</code>) , cerrarConexion()	Usa los cuatro servicios
Solicitud / Respuesta	Objetos serializables que viajan por el socket. La solicitud indica la operación (<code>TipoOperacion</code>) y los datos; la respuesta indica éxito, mensaje y datos	getOperacion(), ok(), error()	Solicitud usa TipoOperacion
ServicioAutenticacion	Valida credenciales, bloquea usuarios inactivos y registra usuarios nuevos	iniciarSesion(), registrarUsuario(), actualizarUsuario() , eliminarUsuario(), cifrarContrasena()	Usa <code>UsuarioDAO</code>
ServicioClientes	Reglas de negocio de clientes (identificación única, campos obligatorios, no eliminar clientes con facturas)	registrar(), actualizar(), eliminar(), buscar(), listar()	Usa <code>ClienteDAO</code>
ServicioInventario	Catálogo y stock. Sección crítica: <code>reservarStock()</code> y <code>ajustarStock()</code> se sincronizan sobre un mismo candado para que dos hilos no descuenten el mismo producto a la vez	registrarProducto() , actualizarProducto () , eliminarProducto(), ajustarStock(), reservarStock(), productosBajoMinim o()	Usa <code>Inventario</code> y <code>ProductoDAO</code>

Clase	Responsabilidad	Métodos principales	Relaciones
ServicioFacturacion	Arma y guarda la factura, reserva el stock, asigna el consecutivo de Factura en un método <code>synchronized</code> y genera el archivo	<code>crearFactura()</code> , <code>anularFactura()</code> , <code>listarFacturas()</code>	Usa <code>ServicioInventario</code> , <code>FacturaDAO</code> , <code>GeneradorArchivoFactura</code>
GeneradorArchivoFactura	Escribe la factura física con su desglose en <code>.txt</code> o <code>.html</code> usando <code>FileWriter/PrintWriter</code>	<code>generarTXT()</code> , <code>generarHTML()</code>	Usado por <code>ServicioFacturacion</code>
ConexionBD (Singleton)	Centraliza la URL y credenciales de la base de datos (MySQL, o Derby si se cambia la URL) y entrega conexiones JDBC	<code>getInstancia()</code> , <code>obtenerConexion()</code>	Usada por todos los DAO
UsuarioDAO , ClienteDAO , ProductoDAO , FacturaDAO	Ejecutan las sentencias SQL (<code>PreparedStatement</code>) de cada entidad	<code>insertar()</code> , <code>listar()</code> , <code>buscarPorId()</code> , <code>actualizar()</code> , <code>eliminar()</code> (CRUD)	Dependen de <code>ConexionBD</code>
ClienteSocket	Del lado del cliente: abre el socket hacia el servidor y envía/recibe objetos	<code>conectar()</code> , <code>enviar(Solicitud)</code> , <code>desconectar()</code>	Usado por todas las ventanas
VentanaLogin (JFrame)	Pide usuario y contraseña; si son válidos abre la ventana principal	<code>btnIngresarActionPerformed()</code>	Abre <code>VentanaPrincipal</code>
VentanaPrincipal (JFrame)	Menú y pestañas de los módulos; oculta los módulos de administrador a los vendedores	<code>mostrarModulo()</code>	Contiene los paneles
PanelClientes , PanelProductos , PanelFacturacion (JPanel)	Pantallas de cada módulo	<code>cargarTabla()</code> , <code>guardar...</code> (), <code>eliminar...</code> (), <code>emitirFactura()</code>	Usan <code>ClienteSocket</code>
HiloActualizacionStock (SwingWorker)	Refresca la tabla de inventario en segundo plano sin congelar la interfaz	<code>doInBackground()</code> , <code>done()</code>	Usado por <code>PanelProductos</code>

4.5 Flujo concurrente de emisión de una factura

El siguiente diagrama de secuencia muestra cómo se emite una factura y dónde se controla la concurrencia.

Figura 6*Diagrama de secuencia: emitir factura*

4.6 Modelo de la base de datos

La base de datos **fidecompro** guarda todo lo que se hace desde las pantallas. Se diseña para MySQL y funciona igual en Apache Derby. Los tres tipos de producto se guardan en una sola tabla **productos**, con una columna **tipo** y las columnas propias de cada tipo (las que no aplican quedan en **NULL**).

Figura 7

Modelo de la base de datos

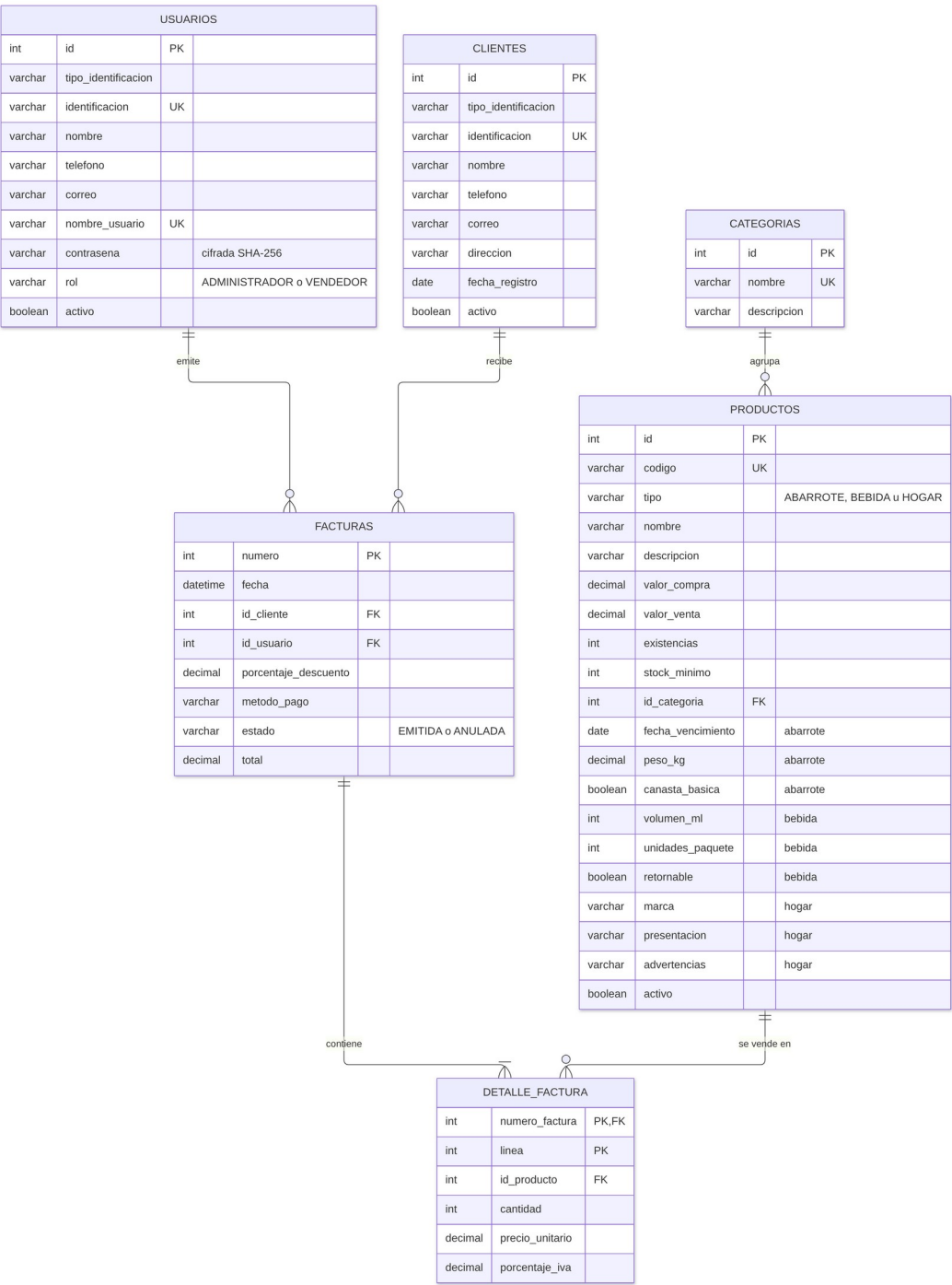


Tabla 30

Tablas de la base de datos

Tabla	Guarda	Llave primaria	Relaciones
usuarios	Cuentas de acceso con su rol y contraseña cifrada	id	Un usuario emite muchas facturas
clientes	Cientes de Fidecompro	id	Un cliente tiene muchas facturas
categorias	Categorías del catálogo	id	Una categoría agrupa muchos productos
productos	Productos de los tres tipos, con existencias	id	Pertenece a una categoría; aparece en muchas líneas de factura
facturas	Encabezado de cada factura	numero	Pertenece a un cliente y a un usuario
detalle_factura	Líneas de cada factura con el precio e IVA del momento	numero_factura + linea	Pertenece a una factura y a un producto

5. Historias de usuario

Cada historia sigue el formato *Como un <rol>, necesito <funcionalidad>, con la finalidad de <resultado>* y se acompaña de sus escenarios de aceptación en la forma **Dado que** (contexto), **Cuando** (evento) y **resultado esperado**. Los escenarios cubren el caso exitoso y los errores que el sistema debe controlar.

Tabla 31

Historias de usuario con sus escenarios de aceptación

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
HU 1	Como un usuario (vendedor o administrador)	Necesito ingresar a la aplicación con mi nombre de usuario y contraseña, y cerrar la sesión al terminar de trabajar	Con la finalidad de que solo el personal autorizado pueda facturar y modificar el inventario, y que nadie más use mi cuenta en la misma caja	1	Inicio de sesión exitoso.	Dado que el usuario está activo y escribe su nombre de usuario y su contraseña correctos en la pantalla de inicio de sesión.	Cuando presiona el botón "Ingresar".	El sistema valida las credenciales, abre el menú principal y muestra en la barra de estado el nombre del usuario y su rol (ADMINISTRADOR o VENDEDOR).
				2	Datos de ingreso inválidos.	Dado que el usuario deja vacío algún campo, escribe un usuario o una contraseña incorrectos, o su cuenta fue desactivada.	Cuando presiona el botón "Ingresar".	El sistema no permite el ingreso. Si hay campos vacíos los resalta y muestra "Debe ingresar usuario y contraseña."; en los demás casos muestra "Usuario o contraseña incorrectos." sin indicar qué dato falló y limpia el campo Contraseña.
				3	Tres intentos fallidos seguidos.	Dado que el usuario ya falló dos intentos seguidos de inicio de sesión.	Cuando falla un tercer intento.	El sistema deshabilita el botón "Ingresar" durante 1 minuto y muestra el mensaje "Demasiados intentos fallidos. Intente de nuevo en 1 minuto.".
				4	Cierre de sesión.	Dado que el usuario tiene una sesión abierta.	Cuando selecciona la opción "Cerrar sesión" del menú principal.	El sistema cierra la sesión, libera la conexión con el servidor y muestra la pantalla de inicio de sesión vacía. Si hay una factura sin emitir, primero pregunta "Tiene una factura sin emitir. ¿Desea

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
								descartarla y cerrar sesión?"; si responde "No", vuelve a la factura sin cambios.
HU 2	Como un administrador	Necesito crear, consultar, modificar y eliminar las cuentas de vendedores y administradores	Con la finalidad de controlar quién tiene acceso al sistema	1	Creación exitosa de un usuario.	Dado que el administrador completa nombre completo, tipo y número de identificación (cédula), teléfono, correo, nombre de usuario, contraseña, confirmación de la contraseña y rol. La contraseña tiene al menos 8 caracteres.	Cuando presiona el botón "Guardar".	El sistema crea el usuario con un ID automático y estado Activo, muestra el mensaje "Usuario creado correctamente." y lo agrega a la tabla de usuarios.
				2	Datos inválidos.	Dado que el nombre de usuario o la cédula ya pertenecen a otra cuenta, o la contraseña tiene menos de 8 caracteres o no coincide con el campo "Confirmar".	Cuando presiona el botón "Guardar".	El sistema no guarda el usuario, resalta el campo con error, conserva los demás datos y muestra el mensaje correspondiente: "El nombre de usuario ya existe.", "Ya existe un usuario con esa identificación." o "La contraseña debe tener al menos 8 caracteres y coincidir con la confirmación.".
				3	Eliminar un usuario.	Dado que el administrador selecciona un usuario en la tabla.	Cuando presiona el botón "Eliminar" y confirma la acción.	Si el usuario no ha emitido facturas, el sistema lo borra, muestra "Usuario eliminado correctamente." y lo quita de la tabla. Si ya emitió facturas, no lo borra: avisa "El usuario tiene facturas registradas; se desactivará para conservar el historial." y lo deja Inactivo, sin poder iniciar sesión.
				4	Acceso de un vendedor.	Dado que inició sesión un usuario con rol VENDEDOR.	Cuando revisa las opciones del menú principal.	El sistema no muestra la opción "Usuarios", de modo que el vendedor no puede entrar a este módulo.
HU 3	Como un vendedor (y como administrador para eliminar)	Necesito registrar, buscar, modificar y eliminar clientes con sus datos de identificación y de	Con la finalidad de poder emitirles facturas con su información al día, sin perder el	1	Registro exitoso con datos válidos.	Dado que el vendedor completa tipo de identificación, número de identificación, nombre o razón social, teléfono,	Cuando presiona el botón "Guardar".	El sistema valida la información, crea el cliente con un ID automático y la fecha de registro del día, muestra el mensaje "Cliente

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
		contacto	historial de facturas			correo y dirección. Tipo, identificación y nombre son obligatorios. La cédula física tiene formato X-XXXX-XXXX y la jurídica X-XXX-XXXXXX.		registrado correctamente." y lo agrega a la tabla de clientes.
				2	Datos inválidos o repetidos.	Dado que falta el tipo de identificación, la identificación o el nombre; la identificación o el correo no tienen el formato correcto; o la identificación ya pertenece a otro cliente.	Cuando presiona el botón "Guardar".	El sistema no guarda el cliente, resalta los campos con error, conserva los demás datos y muestra el mensaje correspondiente, por ejemplo "Debe completar los campos obligatorios.", "Formato de correo electrónico inválido." o "Ya existe un cliente con la identificación indicada.".
				3	Búsqueda y edición.	Dado que el vendedor escribe parte del nombre o de la identificación (por ejemplo, "esquina"), selecciona un cliente de los resultados y modifica su teléfono, correo o dirección.	Cuando presiona "Buscar" y luego "Guardar".	El sistema muestra solo los clientes que contienen el texto, sin distinguir mayúsculas (o "No se encontraron clientes." si no hay ninguno); al guardar actualiza el cliente y muestra "Cliente actualizado correctamente.". Con "Limpiar" descarta los cambios.
				4	Eliminar un cliente.	Dado que el administrador selecciona un cliente en la tabla (para un VENDEDOR el botón "Eliminar" está deshabilitado).	Cuando presiona el botón "Eliminar" y responde "Sí" a la confirmación.	Si el cliente no tiene facturas, el sistema lo borra, muestra "Cliente eliminado correctamente." y lo quita de la tabla. Si tiene facturas, no lo borra: lo desactiva para conservar el historial y deja de aparecer en Nueva factura. Si responde "No", no hay cambios.
HU 4	Como un administrador	Necesito crear, editar y eliminar las categorías de productos (abarrotes, bebidas, limpieza, higiene...)	Con la finalidad de mantener el catálogo ordenado	1	Creación exitosa de una categoría.	Dado que el administrador escribe el nombre y la descripción de una categoría nueva en la pestaña Categorías.	Cuando presiona el botón "Guardar".	El sistema crea la categoría con un ID automático, muestra el mensaje "Categoría creada correctamente." y la agrega a la lista y al filtro de productos.
				2	Nombre de categoría repetido.	Dado que ya existe una categoría con el mismo nombre.	Cuando presiona el botón "Guardar".	El sistema no crea la categoría y muestra el mensaje "Ya existe una categoría con ese

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
				3	Eliminar una categoría.	Dado que el administrador selecciona una categoría en la lista.	Cuando presiona el botón "Eliminar" y confirma la acción.	Si la categoría no tiene productos, el sistema la elimina y la quita de la lista y del filtro de productos. Si tiene productos, no la elimina y muestra "No se puede eliminar la categoría porque tiene productos."
HU 5	Como un administrador	Necesito registrar, consultar, modificar y eliminar productos indicando su tipo (abarrotes, bebida o artículo del hogar)	Con la finalidad de poder venderlos y controlar sus existencias	1	Registro exitoso de un abarrotes.	Dado que el administrador elige el tipo Abarrotes y completa código, nombre, categoría, valor de compra, valor de venta, existencias, stock mínimo, fecha de vencimiento, peso y si es de canasta básica.	Cuando presiona el botón "Guardar".	El sistema crea el producto con un ID automático, le asigna IVA de 1 % si es de canasta básica o de 13 % si no lo es, muestra el mensaje "Producto registrado correctamente." y lo agrega a la tabla ordenada por nombre.
				2	Cambio del tipo de producto.	Dado que el administrador está llenando el formulario de un producto nuevo.	Cuando cambia el tipo de producto a Bebida.	El sistema cambia el recuadro de datos específicos para pedir volumen (ml), unidades por paquete y si es retornable, e indica que el producto pagará IVA de 13 %.
				3	Datos inválidos o repetidos.	Dado que el código ya pertenece a otro producto, el valor de venta es 0 o menor que el de compra, o las existencias son negativas.	Cuando presiona el botón "Guardar".	El sistema no guarda el producto, resalta el campo con error y muestra el mensaje correspondiente, por ejemplo "Ya existe un producto con ese código." o "El valor de venta debe ser mayor que 0 y no menor que el valor de compra."
				4	Eliminar un producto.	Dado que el administrador selecciona un producto que no aparece en ninguna factura.	Cuando presiona el botón "Eliminar" y confirma.	El sistema borra el producto de la base de datos y de la tabla. Si el producto ya aparece en facturas, no lo borra: lo desactiva para que no se pueda vender y muestra el motivo.
HU 6	Como un administrador	Necesito registrar entradas de	Con la finalidad de que el	1	Ajuste válido.	Dado que el administrador selecciona	Cuando presiona el botón "Aplicar" y	El sistema actualiza las existencias (68 o 13), muestra

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
		mercadería y ajustes de existencias	inventario del sistema coincida con lo que hay en bodega			un producto con 18 unidades, abre "Ajustar stock...", elige un motivo y escribe la cantidad (por ejemplo, Compra de 50 o Merma de 5).	confirma.	"Existencias actualizadas: N unidades.", refresca la tabla y quita la alerta de stock bajo si ya no aplica.
				2	Ajuste que deja existencias negativas.	Dado que el producto tiene 9 unidades y el administrador registra un ajuste de -12.	Cuando presiona el botón "Aplicar".	El sistema no aplica el ajuste y muestra el mensaje "El ajuste dejaría las existencias en negativo."
				3	Motivo sin indicar.	Dado que el administrador escribe la cantidad pero no elige el motivo del ajuste.	Cuando presiona el botón "Aplicar".	El sistema no aplica el ajuste y muestra el mensaje "Debe indicar el motivo del ajuste."
HU 7	Como un administrador	Necesito ver qué productos están por debajo del stock mínimo	Con la finalidad de reabastecerlos a tiempo	1	Alerta y consulta de productos bajo el mínimo.	Dado que hay productos cuyas existencias son menores o iguales a su stock mínimo.	Cuando el administrador entra al menú principal o abre la pestaña "Bajo el mínimo".	El menú muestra el aviso "N productos bajo el stock mínimo" con sus nombres, y la pestaña los lista de menos a más existencias, marcados en rojo.
				2	Producto reabastecido.	Dado que un producto bajo el mínimo recibe una entrada de mercadería que lo deja por encima del mínimo.	Cuando se aplica el ajuste.	El sistema quita el producto del aviso del menú principal y de la pestaña "Bajo el mínimo".
HU 8	Como un vendedor	Necesito crear una factura seleccionando al cliente y los productos que compra	Con la finalidad de registrar la venta y cobrarle el monto correcto	1	Factura emitida correctamente.	Dado que el vendedor selecciona un cliente activo, agrega productos con cantidades que no superan las existencias, opcionalmente escribe un descuento entre 0 y 100 % y elige el método de pago.	Cuando presiona el botón "Emitir factura".	El sistema calcula subtotal, descuento, IVA de cada línea sobre el monto con descuento según el tipo de producto y total; asigna el número consecutivo (por ejemplo FC-000123); descuenta las existencias y muestra la vista previa de la factura.
				2	Producto agregado dos veces.	Dado que el producto ya está en la factura.	Cuando el vendedor lo agrega de nuevo.	El sistema suma la cantidad a la misma línea (sin pasar de las existencias) y recalcula los totales.
				3	Cantidad mayor a las existencias.	Dado que el vendedor escribe una cantidad mayor a las unidades disponibles del producto.	Cuando presiona el botón "Agregar".	El sistema no agrega la línea y muestra el mensaje "Stock insuficiente de <producto>: hay N unidades."
				4	Factura	Dado que la factura no	Cuando presiona el	El sistema no emite la factura

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
					incompleta o descuento inválido.	tiene cliente, no tiene productos o el descuento está fuera del rango de 0 a 100.	botón "Emitir factura".	y muestra el mensaje correspondiente: "Debe seleccionar un cliente y agregar al menos un producto." o "El descuento debe estar entre 0 y 100."
HU 9	Como un administrador	Necesito que varias cajas puedan facturar al mismo tiempo	Con la finalidad de atender a más clientes sin vender más producto del que hay en inventario	1	Dos cajas con existencias suficientes.	Dado que dos cajas facturan el mismo producto y la suma de las cantidades no supera las existencias.	Cuando ambas emiten su factura casi al mismo tiempo.	El sistema emite las dos facturas con números consecutivos distintos y descuento de las existencias la suma de ambas cantidades.
				2	Dos cajas compiten por las últimas unidades.	Dado que quedan 9 unidades de un producto y cada caja tiene una factura con esas 9 unidades.	Cuando ambas emiten su factura al mismo tiempo.	El sistema emite solo una de las facturas; la otra caja recibe el mensaje "Stock insuficiente de <producto>: hay 0 unidades." y conserva su factura para corregirla. Las existencias nunca quedan en negativo. La siguiente factura emitida recibe el siguiente número consecutivo, sin huecos por la venta rechazada.
HU 10	Como un vendedor	Necesito obtener la factura en un archivo con el desglose de pago	Con la finalidad de entregarla o imprimirla al cliente	1	Generar el archivo de la factura.	Dado que la factura fue emitida (recién o en otro momento, desde el historial).	Cuando el usuario presiona "Guardar archivo (.txt)" o "Regenerar archivo" y elige una carpeta.	El sistema crea el archivo FC-000123.txt con los datos de Fidecompro, número y fecha, vendedor, cliente, cada línea (cantidad, descripción, precio unitario y total), subtotal, descuento, IVA, total y método de pago, y muestra el mensaje "Archivo guardado en <ruta>.". Si se regenera desde el historial, conserva los precios del momento de la venta.
				2	Error al guardar el archivo.	Dado que la carpeta elegida no tiene permisos de escritura o el disco está lleno.	Cuando el vendedor presiona "Guardar archivo (.txt)".	El sistema muestra el mensaje "No se pudo guardar el archivo de la factura."; la factura sigue emitida y el vendedor puede intentarlo en otra carpeta.

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
HU 11	Como un usuario (vendedor o administrador)	Necesito consultar las facturas por rango de fechas, cliente y estado, y anular las emitidas por error	Con la finalidad de dar seguimiento a las ventas y corregirlas devolviendo el producto al inventario	1	Consulta con filtros.	Dado que el usuario indica una fecha inicial y una final válidas y, si quiere, un cliente y un estado (por ejemplo, ANULADA).	Cuando presiona el botón "Consultar".	El sistema lista las facturas que cumplen los filtros con número, fecha, cliente, vendedor, total y estado, y muestra el total vendido del periodo sumando solo las emitidas. Si no hay ninguna, muestra "No hay facturas para los filtros seleccionados."
				2	Rango de fechas inválido.	Dado que la fecha inicial es posterior a la fecha final.	Cuando presiona el botón "Consultar".	El sistema no realiza la consulta y muestra el mensaje "La fecha inicial no puede ser posterior a la final."
				3	Anulación exitosa.	Dado que el administrador selecciona en el historial una factura en estado EMITIDA.	Cuando presiona el botón "Anular factura" y confirma la acción.	El sistema cambia el estado a ANULADA, devuelve al inventario las unidades de cada línea y recalcula el total del periodo sin esa factura.
				4	Anulación no permitida.	Dado que la factura seleccionada ya está ANULADA, o quien consulta es un VENDEDOR.	Cuando intenta anularla.	Si ya está anulada, el sistema no hace cambios y muestra "La factura ya estaba anulada.". Para un VENDEDOR el botón "Anular factura" no aparece, porque solo el administrador puede anular.
HU 12	Como un administrador	Necesito una ventana del servidor donde pueda iniciarlo, detenerlo y ver los clientes conectados y las operaciones que atiende	Con la finalidad de comprobar que el servidor atiende a varias cajas al mismo tiempo y que todo queda guardado en la base de datos	1	Iniciar el servidor.	Dado que la ventana del servidor está abierta, el puerto es 5000 y la base de datos está disponible.	Cuando el administrador presiona "Iniciar".	El servidor abre el puerto, muestra el estado "En ejecución" y queda esperando clientes.
				2	Base de datos no disponible.	Dado que la base de datos no responde.	Cuando el administrador presiona "Iniciar".	El servidor no se inicia y muestra el mensaje "No se pudo conectar con la base de datos."
				3	Varios clientes conectados y bitácora.	Dado que el servidor está en ejecución.	Cuando se abren tres ventanas cliente, inician sesión y hacen operaciones.	La ventana del servidor muestra las tres conexiones, cada una con su hilo, equipo, usuario y hora, y la bitácora agrega una fila por operación con la hora, el hilo, la operación y su resultado (OK o el error). Al cerrar una ventana cliente, su fila

Identificador (ID) de la historia	Rol	Característica / Funcionalidad	Razón / Resultado	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
				4	Detener el servidor.	Dado que el servidor está en ejecución con clientes conectados.	Cuando el administrador presiona "Detener" y confirma.	desaparece. El servidor cierra las conexiones y el puerto; las ventanas cliente muestran "Se perdió la conexión con el servidor."

Resumen de las historias

Tabla 32

Resumen de las historias de usuario

ID	Historia	Rol	Prioridad	Puntos	Pantalla
HU 1	Iniciar y cerrar sesión	Todos	Alta	4	P1, P2
HU 2	Administrar usuarios	Administrador	Alta	5	P8
HU 3	Administrar clientes	Vendedor (eliminar: administrador)	Alta	7	P3
HU 4	Administrar categorías	Administrador	Alta	2	P4, P10
HU 5	Registrar producto	Administrador	Alta	3	P4
HU 6	Ajustar inventario	Administrador	Alta	3	P4
HU 7	Alertas de stock bajo	Administrador	Media	2	P2, P4
HU 8	Crear factura	Vendedor	Alta	8	P5
HU 9	Facturación simultánea	Administrador	Alta	5	P5
HU 10	Generar factura física	Vendedor	Alta	3	P6
HU 11	Consultar y anular facturas	Todos (anular: administrador)	Media	6	P7
HU 12	Ventana del servidor	Administrador	Alta	5	P9

6. Prototipos de interfaz gráfica

Los prototipos (bocetos o mockups) representan las ventanas JFrame de la aplicación final (look and feel Nimbus): nueve del cliente y una del servidor. Se elaboraron como maquetas HTML/CSS, incluidas en docs/prototipos/html/, y se exportaron a imagen. Los datos mostrados son de ejemplo.

Cada clase que el usuario administra tiene su pantalla para crear, consultar, modificar y eliminar:

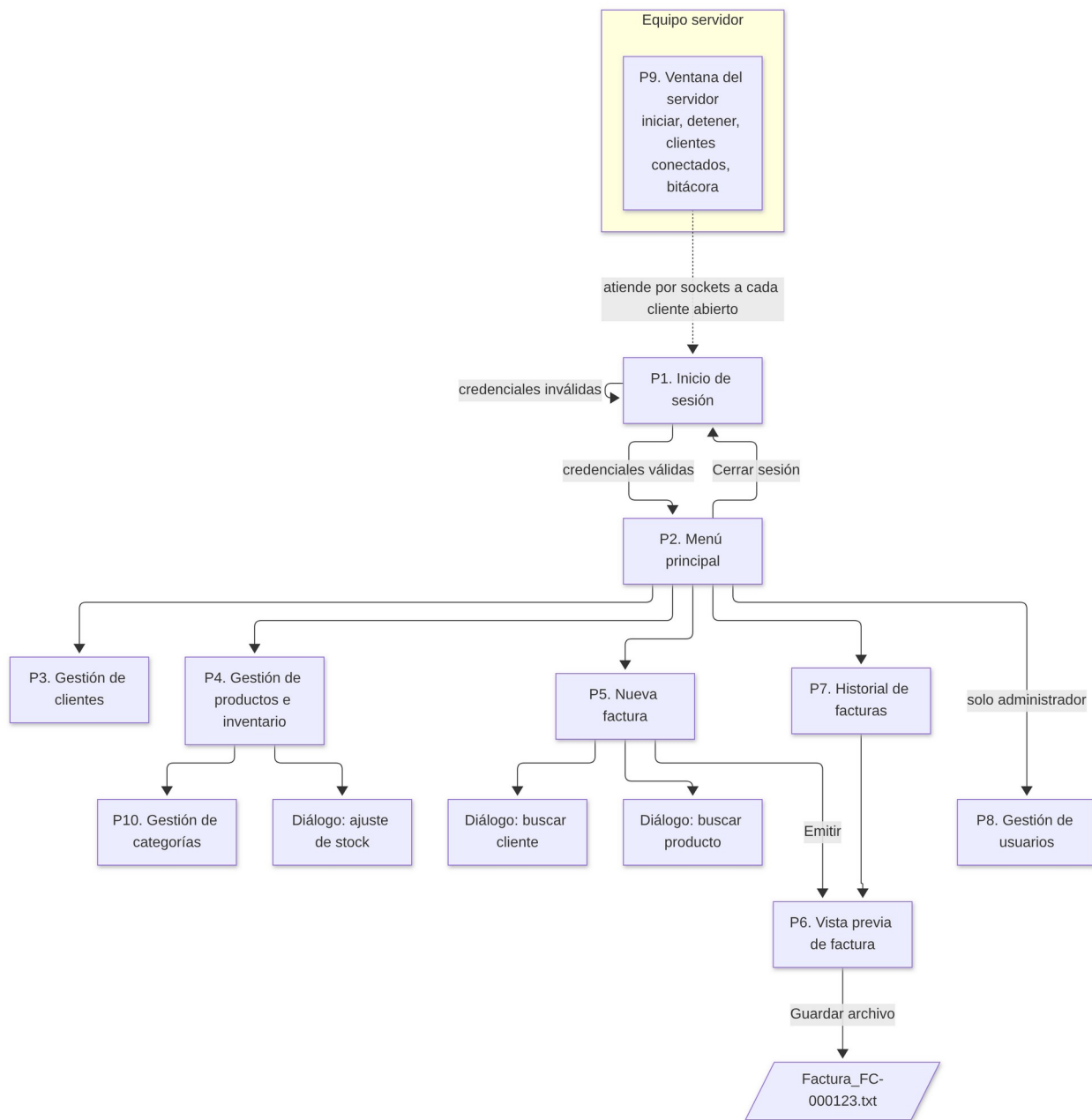
Tabla 33*Pantallas y operaciones CRUD por clase*

Pantalla	Clase que administra	Operaciones
P1 · Inicio de sesión	Usuario	Validar usuario y contraseña
P2 · Menú principal	—	Acceso a todos los módulos según el rol
P3 · Gestión de clientes	Cliente	Crear, consultar, modificar y eliminar
P4 · Productos e inventario	Producto (Abarrote, Bebida, ArtículoHogar) e Inventario	Crear, consultar, modificar, eliminar y ajustar existencias
P10 · Gestión de categorías	Categoria	Crear, consultar, modificar y eliminar
P8 · Gestión de usuarios	Usuario	Crear, consultar, modificar y eliminar
P5, P6 y P7 · Facturación	Factura y DetalleFactura	Crear, ver la factura física, consultar el historial y anular
P9 · Ventana del servidor	ServidorFacturacion	Iniciar, detener y ver clientes conectados

6.1 Mapa de navegación

Figura 8

Mapa de navegación entre pantallas



6.2 P1 · Inicio de sesión (HU 1)

Campos de usuario y contraseña (oculta), dirección del servidor y mensaje de error ante credenciales inválidas.

Figura 9*P1 Inicio de sesión*

The image shows a web application window titled "Fidecompro · Iniciar sesión". The main heading is "FIDECOMPRO" with the subtitle "Sistema de facturación e inventario". There are three input fields: "Usuario:" with the value "mrodriguez", "Contraseña:" with masked characters ".....", and "Servidor:" with the value "192.168.1.10 : 5000". Below these fields is a red error message: "Usuario o contraseña incorrectos (intento 1 de 3)". At the bottom of the form area are two buttons: "Ingresar" (highlighted in blue) and "Salir". The footer bar contains the text "Estado: conectado al servidor" on the left and "v1.0" on the right.

6.3 P2 · Menú principal (HU 1 y HU 7)

Acceso a todos los módulos mediante menú y botones, resumen del día y alerta de productos bajo el stock mínimo. La barra de estado muestra el usuario, su rol y el estado de la conexión. El botón *Usuarios* solo aparece para administradores.

Figura 10

P2 Menú principal



6.4 P3 · Gestión de clientes (HU 3)

CRUD completo de clientes: formulario para crear y modificar, búsqueda, tabla de clientes y botón *Eliminar*. El botón *Actualizar lista* vuelve a pedir los datos al servidor para ver los cambios hechos desde otras cajas.

Figura 11*P3 Gestión de clientes*

Fidecompro · Gestión de clientes

Datos del cliente

Tipo de identificación: Jurídica Identificación: 3-101-456789

Nombre / Razón social: Abastecedor La Esquina S.A. Teléfono: 2222-3344

Correo: compras@laesquina.cr Dirección: Heredia, San Pablo, 200 m norte

Activo: ☒ (desmarcar para ocultarlo al facturar)

Nuevo Guardar Limpiar Eliminar Actualizar lista

Buscar: esquina Buscar

Identificación	Nombre	Teléfono	Correo	Registro
3-101-456789	Abastecedor La Esquina S.A.	2222-3344	compras@laesquina.cr	12/09/2026
1-1234-0567	Pulpería Doña Ana	8888-1020	ana.pulperia@gmail.com	20/09/2026
3-102-778899	Minisúper El Ahorro Ltda.	2560-7788	admin@elahorro.cr	01/10/2026

3 clientes encontrados Usuario: María Rodríguez

6.5 P4 · Productos e inventario (HU 5, HU 6 y HU 7)

Pestañas para productos, categorías y productos bajo el mínimo. El formulario cambia según el tipo de producto elegido (abarrotes, bebida o artículo del hogar). Los productos bajo el mínimo se resaltan en rojo. Los botones permiten crear, modificar, eliminar y ajustar el stock.

Figura 12

P4 Productos e inventario

Fidecompro · Productos e inventario

Productos Categorías Bajo el mínimo (4)

Datos del producto

Tipo de producto: Abarrote Categoría: Abarrotes

Código: ABR-0012 Nombre: Arroz 99% grano entero 2 kg

Valor compra (C): 1 520,00 Valor venta (C): 1 950,00

Existencias: 18 unidades Stock mínimo: 25

Datos de abarrote

Fecha vencimiento: 15/03/2027 Peso (kg): 2,0 ☒ Canasta básica (IVA 1 %)

Este recuadro cambia según el tipo: Bebida (volumen, unidades por paquete, retornable) · Artículo del hogar (marca, presentación, advertencias)

Nuevo Guardar Ajustar stock... Eliminar Actualizar lista

Filtrar por categoría: Todas buscar por código o nombre Buscar

Código	Nombre	Tipo	Categoría	Precio	Stock	Estado
ABR-0012	Arroz 99% grano entero 2 kg	Abarrote	Abarrotes	¢1 950	18	Bajo mínimo
LIM-0230	Detergente en polvo 3 kg	Art. hogar	Limpieza	¢5 400	9	Bajo mínimo
HIG-0045	Papel higiénico 24 rollos	Art. hogar	Higiene	¢8 750	120	Disponible
BEB-0101	Refresco cola 3 L (caja 6)	Bebida	Bebidas	¢9 300	64	Disponible

Ordenado por nombre (Comparable) · pestaña "Bajo el mínimo" ordena por existencias (Comparador) Usuario: María Rodríguez

6.6 P5 · Nueva factura (HU 8 y HU 9)

Selección de cliente, agregado de productos con validación de existencias, tabla de líneas con el IVA de cada tipo de producto, método de pago, descuento y totales calculados.

Figura 13*P5 Nueva factura*

Fidecompro · Nueva factura

Cliente

Identificación: **Abastecedor La Esquina S.A. · 2222-3344**

Agregar producto

Código: Cantidad: Disponible: 64

#	Código	Descripción	Cant.	Precio unit.	IVA	Total línea
1	ABR-0012	Arroz 99% grano entero 2 kg	10	€1 950,00	1%	€19 695,00
2	HIG-0045	Papel higiénico 24 rollos	4	€8 750,00	13%	€39 550,00
3	BEB-0101	Refresco cola 3 L (caja 6)	5	€9 300,00	13%	€52 545,00

Método de pago:

Descuento (%):

Subtotal	€101 000,00
Descuento (2%)	-€2 020,00
Impuesto (IVA)	€10 574,20
TOTAL	€109 554,20

Vendedor: Carlos Jiménez 06/10/2026 10:42

6.7 P6 · Vista previa de la factura física (HU 10)

Contenido exacto del archivo que se genera, con el desglose de pago, y botones para guardarlo como .txt o .html.

Figura 14*P6 Vista previa de factura*

Fidecompro · Factura FC-000123

FIDECOMPRO - VENTAS AL POR MAYOR
Cédula jurídica 3-101-000000 Tel 2000-0000

Factura N.º : FC-000123 Fecha: 06/10/2026 10:43
 Vendedor : Carlos Jiménez
 Cliente : Abastecedor La Esquina S.A.
 Identific. : 3-101-456789 Tel : 2222-3344

Cant	Descripción	P.Unit	Total
10	Arroz 99% grano entero 2 kg	1 950,00	19 500,00
4	Papel higiénico 24 rollos	8 750,00	35 000,00
5	Refresco cola 3 L (caja 6)	9 300,00	46 500,00

Subtotal : 101 000,00
 Descuento 2% : -2 020,00
 IVA : 10 574,20
TOTAL : ₡ 109 554,20

Método de pago: TRANSFERENCIA

¡Gracias por su compra!

Archivo: facturas/FC-000123.txt Estado: EMITIDA

6.8 P7 · Historial de facturas (HU 11)

Filtros por fecha, cliente y estado; total del periodo y acciones para ver el detalle, regenerar el archivo o anular.

Figura 15*P7 Historial de facturas*

Fidecompro · Historial de facturas

Filtros

Desde: Hasta: Cliente: Estado: [Consultar](#)

Número	Fecha	Cliente	Vendedor	Total	Estado
FC-000123	06/10/2026 10:43	Abastecedor La Esquina S.A.	Carlos Jiménez	€109 554,20	EMITIDA
FC-000122	06/10/2026 09:15	Pulpería Doña Ana	Carlos Jiménez	€34 870,00	EMITIDA
FC-000121	05/10/2026 16:02	Minisúper El Ahorro Ltda.	María Rodríguez	€212 400,00	ANULADA
FC-000120	05/10/2026 11:30	Abastecedor La Esquina S.A.	Carlos Jiménez	€88 015,50	EMITIDA

Total del periodo (emitidas): €232 439,70

[Ver detalle](#) [Regenerar archivo](#) [Anular factura](#)

4 facturas Usuario: María Rodríguez

6.9 P8 · Gestión de usuarios (HU 2)

Exclusiva del administrador: creación, consulta, modificación (incluido el rol y el estado activo), eliminación y restablecimiento de contraseña de usuarios.

Figura 16*P8 Gestión de usuarios*

Fidecompro · Gestión de usuarios (Administrador)

Datos del usuario

Tipo de cédula: Identificación:

Nombre completo: Usuario:

Contraseña: Confirmar:

Rol: Activo: ☒

Usuario	Identificación	Nombre	Rol	Estado	Último ingreso
mrodriguez	1-0987-0654	María Rodríguez Solís	ADMINISTRADOR	Activo	06/10/2026 08:01
cjimenez	2-0456-0789	Carlos Jiménez Mora	VENDEDOR	Activo	06/10/2026 08:20
lvargas	1-1122-0334	Laura Vargas Pérez	VENDEDOR	Inactivo	14/09/2026 17:45

Las contraseñas se guardan cifradas (SHA-256) Usuario: María Rodríguez

6.10 P9 · Ventana del servidor (HU 12)

Se ejecuta en el equipo servidor, aparte de las cajas. Permite elegir el puerto, iniciar y detener el servidor, y muestra los clientes conectados (cada uno con su hilo) y una bitácora con cada operación que llega y su resultado. Es la pantalla que demuestra en el video que el servidor atiende a varios clientes a la vez.

Figura 17*P9 Ventana del servidor*

Servidor Fidecompro

Servidor

Puerto: Base de datos:

Estado: ● En ejecución desde 08:00:12 Clientes conectados: **3** Hilos activos: **3** de 10

Clientes conectados

Hilo	Equipo (IP)	Usuario	Rol	Conectado desde
Hilo-1	192.168.1.21 (Caja 1)	cjimenez	VENDEDOR	08:02:40
Hilo-2	192.168.1.22 (Caja 2)	apineda	VENDEDOR	08:05:11
Hilo-3	192.168.1.10 (Administración)	mrodriguez	ADMINISTRADOR	08:01:03

Bitácora de operaciones

Hora	Hilo	Operación	Resultado
08:41:05	Hilo-1	CREAR_FACTURA FC-000123 (cliente 3-101-456789)	OK
08:41:05	Hilo-2	CREAR_FACTURA (ABR-0012 × 9)	Stock insuficiente
08:39:47	Hilo-3	ACTUALIZAR_PRODUCTO LIM-0230	OK
08:38:20	Hilo-3	ELIMINAR_CLIENTE 1-0987-0654	OK
08:36:02	Hilo-2	CREAR_CLIENTE 3-102-778899	OK

Cada cliente conectado se atiende en su propio hilo Datos guardados en la base de datos

6.11 P10 · Gestión de categorías (HU 4)

Se abre desde la pantalla de productos. CRUD de categorías con su nombre y descripción; la tabla indica cuántos productos tiene cada una, porque una categoría con productos no se puede eliminar.

Figura 18

P10 Gestión de categorías

Fidecompro · Gestión de categorías (Administrador)

Datos de la categoría

ID:

4

Nombre:

Higiene

Descripción:

Papel higiénico, jabón, pasta dental

Nuevo

Guardar

Eliminar

Actualizar lista

ID	Nombre	Descripción	Productos
1	Abarrotes	Arroz, frijoles, aceite, café, azúcar	42
2	Bebidas	Refrescos, jugos y agua	18
3	Limpieza	Detergentes, cloro y desinfectantes	15
4	Higiene	Papel higiénico, jabón, pasta dental	11

Una categoría que todavía tiene productos no se puede eliminar.

4 categorías

Usuario: María Rodríguez

7. Trazabilidad historias – clases – pantallas

Tabla 34

Trazabilidad entre historias, clases y pantallas

Historia	Clases principales	Pantalla
HU 1	Usuario, RegistroUsuarios, CredencialesInvalidasException, VentanaLogin	P1, P2
HU 2	Usuario, Rol, RegistroUsuarios	P8
HU 3	Persona, Cliente, RegistroClientes, RegistroNoEncontradoException	P3
HU 4 y HU 5	Inventario, Categoria, Producto, Abarrote, Bebida, ArticuloHogar	P4, P10
HU 6 y HU 7	Producto, Inventario, ComparadorPorExistencias	P2, P4
HU 8 y HU 9	Factura, DetalleFactura, Inventario, StockInsuficienteException (y en la arquitectura prevista: ServicioFacturacion, ManejadorCliente)	P5
HU 10	Factura, DetalleFactura, Mostrable	P6
HU 11	Factura, EstadoFactura, RegistroFacturas	P7
HU 12	VentanaServidor,	P9

Historia	Clases principales	Pantalla
	ServidorFacturacion, ManejadorCliente	

8. Plan de demostración del proyecto final

El video final debe mostrar todas las funcionalidades, el CRUD completo desde las pantallas, que todo queda guardado en la base de datos y que el servidor soporta varios clientes a la vez. Esta matriz muestra qué operación del CRUD tiene cada entidad y en qué pantalla se hace:

Tabla 35

Operaciones CRUD que se mostrarán en el video

Entidad	Crear	Consultar	Modificar	Eliminar	Pantalla
Usuarios	Sí	Sí	Sí	Sí (si tiene facturas, se desactiva)	P8
Clientes	Sí	Sí	Sí	Sí (si tiene facturas, se desactiva)	P3
Categorías	Sí	Sí	Sí	Sí (si no tiene productos)	P10
Productos	Sí	Sí	Sí	Sí (si está en facturas, se desactiva)	P4
Facturas	Sí	Sí (historial)	No: una factura emitida no se modifica	Se anula, no se borra	P5, P6, P7

Guion previsto para el video:

1. Abrir la ventana del servidor (P9), iniciarlo y mostrar que se conecta a la base de datos.
2. Abrir dos o tres ventanas cliente al mismo tiempo e iniciar sesión con usuarios distintos; mostrar en P9 que cada uno tiene su hilo.
3. Hacer el CRUD de usuarios, clientes, categorías y productos: crear, buscar, modificar y eliminar un registro de cada uno.
4. Después de cada operación, mostrar la tabla correspondiente en la base de datos (MySQL Workbench o la consola de Derby) para comprobar que quedó guardada.
5. Desde otra caja, presionar *Actualizar lista* y mostrar que ve los cambios hechos por la primera.
6. Emitir una factura, ver la vista previa y generar el archivo de la factura física.
7. Facturar el mismo producto desde dos cajas casi al mismo tiempo: una factura sale y la otra recibe el aviso de stock insuficiente, sin existencias negativas.
8. Consultar el historial, anular una factura y mostrar que las existencias vuelven al inventario.
9. Cerrar todo, volver a abrir el servidor y un cliente, y mostrar que los datos siguen ahí.